

Your role is as architect and senior software engineer, below is consolidated the technical requirements, the revised phase-by-step roadmap, and the specific package details into a single, unified master blueprint for **MangoCMS**.

Directory name is mangocms and module name is MangoCMS.

This plan respects our "local-first" development strategy, the use of Redis for the initial caching layer, and the strict non-umbrella architecture.

The Architectural Master Blueprint

Core Stack & Package Manifest

Category	Technology	Package Details
Backend Core	Phoenix 1.8.5 (Elixir 1.19)	phoenix, phoenix_live_view
Caching (P1)	Redis (Local/Cloud)	redis ~> 1.5
Caching (P4)	Go Sidecar	Custom binary (5 ops: Resolving, Session, Rate-limit, Page Cache, Job-dedup)
Primary DB	SQLite (Direct/WAL)	exqlite ~> 0.20
Micro DB	JSON Serialization	jason ~> 1.4
Background	Oban	oban ~> 2.17
Mobile	Flutter SDK	flutter_riverpod, dio, hive, phoenix_socket
Infrastructure	Fly.io + Hetzner	Mumbai Region (bom), Hetzner StorageBox (rsync/SSH)

Execution Roadmap (Phase-by-Phase)

PHASE 1 — Foundation (Weeks 1-4)

Goal: Platform boots, tenant resolves via Redis, and content is readable from SQLite/JSON.

- **Initialization:** Standard Phoenix project (non-umbrella) named `mangocms` with module `MangoCMS`.
- **Meta DB:** SQLite registry (`platform.db`) to map domains to storage paths.
- **Tenant Resolution:** Middleware (Plug) that checks Redis first, then falls back to Meta DB.
- **Storage Abstraction:** Implement `ContentRepository` behavior.
- **Dual Backends:** Implement `SQLiteBackend` (with FTS5 search) and `JSONBackend` (atomic writes).
- **Security:** Basic web session auth and local dev directory structure (`/data`).
- **Backups:** Oban worker for daily SQLite snapshots synced to Hetzner.

PHASE 2 — CMS Admin (Weeks 5-8)

Goal: A functional back-office for client content management.

- **Admin UI:** LiveView dashboard for real-time site management.

- **Content Editors:** Rich text/block editor for Pages and Posts.
- **Media Engine:** Local uploads with `mogrify` / `libvips` processing.
- **Identity:** Invite-only user management with Role-Based Access Control (RBAC).
- **Provisioning:** Automated script to create new tenant directories and databases.

PHASE 3 — Public Website + SEO (Weeks 9-11)

Goal: SEO-ready public sites live for all tenants.

- **Rendering:** Optimized Public LiveView pages (SSR).
- **Search:** Global search implementation using SQLite `MATCH` against FTS5 indexes.
- **SEO:** Per-tenant `sitemap.xml`, `robots.txt`, and JSON-LD structured data.
- **Traffic Management:** Redirect plug for 301/302 management and custom 404 pages.

PHASE 4 — Cloud & Go Optimization (Weeks 12-14)

Goal: Move to Fly.io and replace Redis with the high-performance Go sidecar.

- **Infrastructure:** Fly.io setup with persistent volumes and Mumbai region routing.
- **Go Cache Development:** Build and integrate the custom Go binary for ultra-low latency.
- **Nginx:** Configure sidecar Nginx to serve media directly from the volume, bypassing Elixir.
- **Cloudflare:** DNS integration and SSL termination.

PHASE 5 — Mobile API & Channels (Weeks 15-16)

Goal: Secure API layer for the Flutter mobile application.

- **Auth:** JWT implementation (Access + Refresh tokens).
- **REST API:** Full content endpoints for mobile consumption.
- **Real-time:** Phoenix Channels for live updates between CMS and App.
- **Rate Limiting:** Throttle API requests per tenant using the Go cache.

PHASE 6 — Flutter App (Weeks 17-21)

Goal: Mobile management app live on App Store & Play Store.

- **Core UI:** Auth screens, dashboard, and media library management.
- **Offline Mode:** Hive local cache for editing content without internet.
- **Native Features:** Push notifications via FCM and direct media uploads.

PHASE 7 — Polish & Production (Weeks 22-24)

Goal: Hardened, production-ready SaaS environment.

- **Observability:** AppSignal for errors and Better Uptime for monitoring.
- **Testing:** k6 load testing and security audits.
- **Onboarding:** Successful migration of the first real-world tenant.

Internal Verification (Architect's Checklist)

1. **Isolation:** Are tenant databases strictly separated in `/data/tenants/{id}/`? **Yes.**
2. **Performance:** Does the resolution use the Go/Redis cache before hitting SQLite? **Yes.**
3. **Efficiency:** Does Nginx serve media to save Phoenix resources? **Yes.**
4. **Resilience:** Are daily backups automated via Oban to Hetzner? **Yes.**